# CSE160 Homework 4

Elijah Hantman

May 17, 2026

## View and Projection

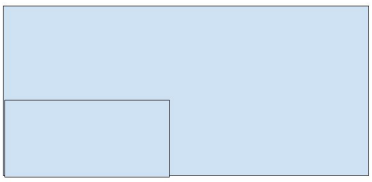
1. What matrix is created by LookAt(eye=[0,0,2], at=[0,0,0], up=[0,1,0])? Is there a way to get this matrix using translate and/or rotate and/or scale? Explain

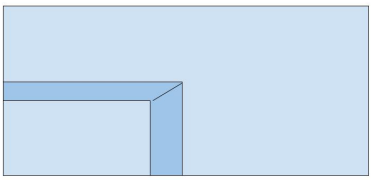
   > The matrix places the point $0, 0, 2$ at the origin, and orients the transformed origin point $0, 0, 0$ to sit at $0, 0, 1$. Finally the up vector ensures that the point $0, 1, 2$ ends up at $0, 1, 0$.
   >
   > To replicate this matrix first you translate $0, 0, 2$ to the origin. Then you rotate so that $0, 1, 0$ lands on $0, 1, 0$ and $0, 0, -2$ ends up at $0, 0, 2$.

2. Illustrate the difference between orthographic (parallel) and perspective projection.

   > The main difference between perspective projection and orthographic projection is that orthographic projection places a constant in the w component while the perspective projection puts the depth in the w component. This means that when the hardware does a perspective divide the orthographic projection does not have perspective while the perspective projection makes objects further ways smaller.
   >
   > Conceptually we can imagine the region each projection will put into the unit cube to be rendered. For orthographic projections this region is a rectangular prism meaning things further away from the origin are projected to the same size as things close by.
   >
   > On the other hand, perspective projections have whats called a frustum which is a cut off rectangular pyramid. When the shape is transformed into a unit cube the things further away are shrunk down compared to things near by.

3. Consider the 3D solid cube, with a cube-shaped notch removed from one corner.

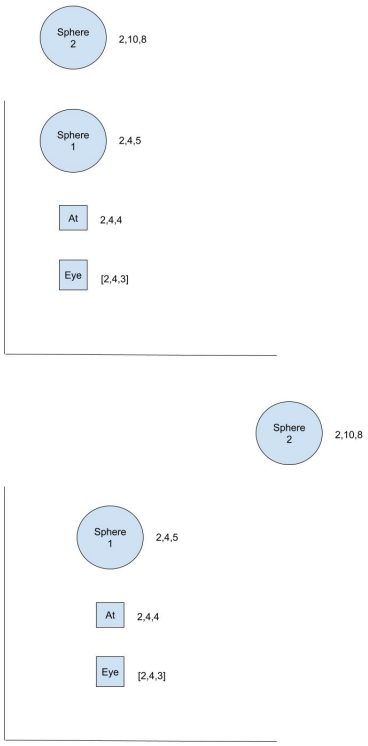




> The top is orthographic, as can be seen by the walls being parallel to the camera view.
>
> The second is perspective and that can be seen by the walls not being parallel due to perspective distortion.

4. Suppose we set up a camera using LookAt eye=[2,4,3], at=[2,4,4], up=[0,1,0] and render a scene with two spheres of radius 1, centered at [2,4,5] and [2,10,8].

(a) Draw this scene in 2D both on the x-z plane (left) and on the y-z plane (right). Label the eye and the at points as well as the two spheres.




(b) What is the distance to the near and far plane if they are set to bound the objects as closely as possible? Describe the distance in numerical values and explain it.

The near plane must be close enough that no part of the closest object crosses it, and the far plane must be far enough that no part of the farthest object crosses it.

For this scenario a near plane of 1 would make sense since the near sphere is 2 units from the eye, and has a radius of 1.

The far plane would need to be set to 6, since it is 5 units away long the look direction and 1 unit in radius.

We measure distance along the look direction since this is what is used to define the near and far planes rather than the euclidean distance between the points.

(c) If we render this scene with a projection matrix, how many spheres will be seen in each of the following cases? Why?

i. perspective(fovy=1°, aspect=1, zNear=1, zFar=100)

> You would see only the first sphere. It is within the near and far planes so that isn't an issue.
> However the 1 degree field of view means that only a sphere directly in line with or very close to the look direction will be visible. The other sphere is more than 30 degrees away from the look direction meaining it won't be visible.

ii. perspective(fovy=175°, aspect=1, zNear=4, zFar=10)

> In this case we would only see the far sphere. The near sphere is closer than the nearplane, and the fov is wide enough that the far sphere is captured in it. In addition the far plane is far enough away to capture the far sphere.

iii. perspective(fovy=60°, aspect=1, zNear=1, zFar=10)

> I believe both spheres would be visible. The near plane and far plane are far enough apart that both spheres lie between them, and the fov is high enough that the far sphere should be within the view frustum.

## Lighting

1. Define the three components of light used in the Phong lighting model: diffuse, specular, and ambient.

> The diffuse component is the light absorbed and scattered from an object in all directions. This happens when light hits a rough surface and bounces seemingly randomly.
>
> The specular component is the light which bounces in one particular direction. This happens when light hits a smooth surface and reflects.
>
> The ambient light is a representation of the light which bounces many times off the objects in the environment.
>
> In the lighting model the diffuse component is modeled via the cosine, the specular via the reflection, and the ambient is a constant added to the base light.

2. Match the images below to the properties defined in the table. Write the image number in the correspondent table row.

| Diffuse Color | Specular Color | Specular Exponent | Image Number |
|---|---|---|---|
| Red | Black | Any | 7 |
| Blue | Red | Small | 3 |
| Blue | White | Big | 1 |
| Dark Blue | Red | Medium-big | 10 |
| Blue | Red | Big | 4 |
| Blue | Black | Any | 2 |
| Black | Red | Big Red | 9 |
| Red | White | Small | 6 |
| Red | White | Big | 8 |
| Red | Blue | Big | 5 |

I am assuming that by big the table means that the p value is small, since the specular value is between 0 and 1, (clamped) raising it to a power will actually lower the value.

3. Consider a scene with a single sphere of radius 1 centered at the origin (0,0,0). The sphere's ambient coefficient is ka = RGB(0.1,0.1,0.1), diffuse coefficient kd = RGB(0.5,0.5,0.5), and specular coefficient ks = RGB(0.5,0.5,0.5), with specular ex- ponent s = 2. There is a light at position p1 = (0,2,0) with color c1 = RGB(1.0, 0.0, 0.0). The camera is located at position p2 = (3,1,0). Calculate the Phong Illumination for the sphere fragment located at position p3 = (0,1,0).

   (a) Calculate the ambient portion of the color of this fragment.

   The ambient coefficient is 0.1, meaning the ambient light is $RGB(0.1, 0.1, 0.1) * RGB(1.0, 0.0, 0.0) = RGB(0.1, 0.0, 0.0)$.

   (b) Calculate the diffuse portion of the color of this fragment.

   The diffuse component is calculated by taking the dot product between the light direction and the normal, then multiplying by the diffuse coefficient and the lighting color.

   $$n = [0, 1, 0]$$
   $$l = [0, 2, 0] - [0, 1, 0] = [0, 1, 0]$$
   $$n \cdot l = 1$$
   $$c_l c_r = RGB(0.5, 0.0, 0.0)$$

   (c) Calculate the specular portion of the color of this fragment.

$$h = \frac{e + l}{||e + l||}$$

$$h = \frac{[3, 0, 0] + [0, 1, 0]}{||e + l||}$$

$$h = [3, 1, 0]/\sqrt{10}$$

$$(h \cdot n)^p = (1/\sqrt{10})^2 = 1/10 = 0.1$$

$$c = c_l * 0.1 = RGB(0.1, 0.0, 0.0)$$

(d) Calculate the total color of this fragment.

Since the terms were written as ambient and diffuse coefficient, I will separate them out rather than add the diffuse and ambient terms before multiplying by the diffuse coefficient.

$$c = c_a + c_d + c_s = RGB(0.7, 0.0, 0.0)$$

(e) Now suppose the camera moves to p4 = (0,3,0). What is the total color of the fragment?

Only the specular would change.

$$h = \frac{e + l}{||e + l||}$$

$$h = \frac{[0, 2, 0] + [0, 1, 0]}{||e + l||}$$

$$h = [0, 3, 0]/3$$

$$(h \cdot n)^p = (1)^2 = 1$$

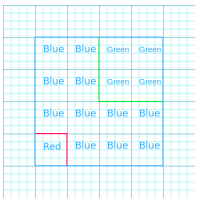
$$c = c_l * 0.1 = RGB(1.0, 0.0, 0.0)$$

Meaning the new color is

$$c = c_a + c_d + c_s = RGB(1.6, 0.0, 0.0) = RGB(1.0, 0.0, 0.0)$$

4. Consider an orthographic scene with objects listed below being rendered at 5 x 5 pixels with with the view plane at z=0 (near=0, far=99). Show the state of the z-buffer (with numbers) once these squares have been rendered (without using anti-aliasing or sub-sampling). Note that the pixels are drawn such that the z-buffer values are stored halfway between integers (look at the labels below the z-buffer).

   Red square with vertices: (0,0,1), (1,0,1), (1,1,1), (0,1,1) Green square with vertices: (2,2,2), (4,2,2), (4,4,2), (2,4,2) Blue square with vertices: (0,0,3), (4,0,3), (4,4,3), (0,4,3)




| Here red refers to a value of 1, Blue a value of 3, and Green a value of 2. Everything else has a value of 99. |
|---|

5. Consider the following vertex and fragment shaders:

```
// Vertex Shader
uniform mat4 u_NormalMatrix;
uniform mat4 u_ModelMatrix;
uniform mat4 u_ViewMatrix;
uniform mat4 u_ProjectionMatrix;
attribute vec4 a_Position;
varying vec3 v_Position;
attribute vec4 a_Normal;
varying vec3 v_Normal;
void main() {
    v_Normal = normalize(vec3(u_NormalMatrix * a_Normal));
    v_Position = vec3(u_ModelMatrix * a_Position);
    gl_Position = u_ProjectionMatrix *
    u_ViewMatrix *
    u_ModelMatrix *
    a_Position;
};

// Fragment Shader
varying vec3 v_Position;
varying vec3 v_Normal;
uniform vec3 u_LightPos;
uniform vec3 u_LightColor;
void main() {
    vec3 l = normalize(u_LightPos - v_Position);
    float nDotL = max(dot(l, v_Normal), 0.0);
    gl_FragColor = vec4(u_LightColor * nDotL);
}
```

(a) In the vertex shader, what is the normal matrix and why do we need to multiply it by the normal vector?

> The normal matrix is the model matrix but without translation. We use it to ensure the normals are transformed to match the models orientation.

(b) In the vertex shader, why do we multiply the vertex position by the model matrix?

> The model matrix will convert the local coordinates of the object relative to its center to the world coordinates. Basically the model matrix stores where the object is and how it is oriented relative to all the other objects in the world, and we use

it to convert our vertex positions to be relative to all the other objects.

(c) What is the fragment shader computing?

The fragment shader computes the light direction, then takes the dot product with the surface normal. This is a lambertian diffuse lighting component meaning it is computing diffuse lighting.

## Readings

1. What are the three items of information required to specify the viewing direction? Describe each one of them.

   The first is the eye position. This is the position of the camera and the origin of our view.

   The second is the target to look at. Usually this is a world space point that the camera will look at, but this could also just be a look direction which gets added to the camera position. This specifies what line will lie in the center of the camera's view.

   The first two together actually leave a degree of freedom to rotate, so the last component specifies the up direction. Essentially if we rotate the view about the line specifying the look direction the first two parameters are satisfied, but by taking a direction orthogonal (ish), to the look direction we can map that to some static point and specify a single linear transformation.

   Essentially the orthogonal component of the up direction will get mapped to the top of the screen.

2. Define a view matrix and describe the method that can be used (according to the book) to calculate it in Javascript?

   A view matrix is a series of translations and rotations which places the eye at the origin, the point 1 unit in the look direction at the point $0, 0, 1$ and the up direction at $0, 1, 0$.

   In javascript we can use the provided lookAt function, which takes an eye position, an up vector, and an at point which is specified in world space, ie: at = eye + look direction.

   ```
   var mat = new Matrix4();
   mat.setLookAt(ex, ey, ez,
                 at_x, at_y, at_z,
                 up_x, up_y, up_z);
   ```

3. What is the difference between orthographic projection and perspective projection? What methods can be used to calculate the orthographic projection? And the perspective projection? Explain the arguments of both the methods.

   Orthographic projections do not divide by the depth, meaning planes parallel to the camera view direction stay parallel in the final image.

   Perspective projections divide by the depth, meaining objects further away from the camera look smaller and straight lines can become curved.

   Orthographic projections are generally specified by giving a bounding box in terms of left, right, near, and far planes. You could do something like aspect ratio plus near and far but this is fairly uncommon since orthographic projections are usually used with a top left origin.

   Perspective projections are usually specified using fov, aspect ratio, near, and far. The fov, alongside the near and far planes are used to specify how dramatic the perspective correction will be. It is possible to specify a perspective frustum using left, right, near and far, but this is uncommon since changing the near and far planes will change the resulting image in an unintuitive way.

   Generally speaking perspective will produce more immersive and realistic images, while orthographic projections are used when precision and exact sizing is needed. So orthographic is the default in many CAD programs, while perspective is the default in game engines.

4. Describe the three major types of light sources: directional, point and ambient.

   Light is defined by the direction from the surface to the light source.

   Directional lights use a constant direction, these lights act like points infinitely far away.

   Point lights are points in space. meaning the direction changes as you move the surface.

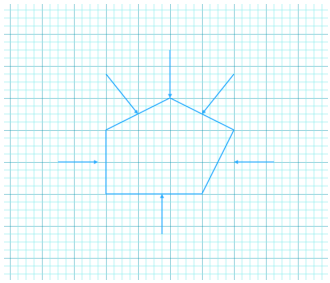
   Ambient light is the light which we imagine has bounced off of many objects and suffuses the scene. This is usually done by just adding a constant light value to the final light. This type of light has no direction and is constant for all surfaces.

5. Explain ambient reflection. Provide the equation that models this reflection and draw a figure illustrating it.

Ambient reflection can be modeled as a constant.

$$c = c_a$$

The diagram shows light hitting the shape from all directions equally.



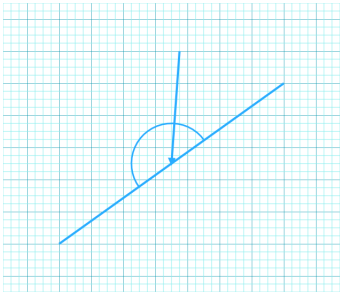
6. Explain diffuse reflection. Provide the equation that models this reflection and draw a figure illustrating it.

Diffuse reflection is the light which hits the surface and scatters equally in all directions. It is proportional only the the amount the surface faces the light.

$$c = c_r(n \cdot l)$$

Where $c_r$ is the diffuse color, $n$ is the surface normal, and $l$ is the light direction.

The image shows light hitting a surface and scattering in all directions.



7. What is a surface normal? How many normals are there in a surface? Draw a cube and the normals of all of its vertices.

The derivative of a surface at a point gives a plane. This plane is effectively the normal of the surface.

This is not obvious at first since we also see the normal vector as a vector which points directly away from the surface, however for every 3D vector there is exactly 1 3D plane.

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} + 1 = 0$$

Is equivalent to

$$ax + by + cz + 1 = 0$$

Which is a general plane equation.

For any given surface we can take the normal at any point, however for surfaces which are composed of triangles there is 1 surface normal per triangle.

That means for a cube there are 6 distinct normals corresponding to the 6 distinct planes which make up the polyhedron.

At a vertex we have two options. We could say the normals point directly away from the center of the cube, which implies the triangles are an approximation of some smooth shape, or we can say that there are multiple normals which are associated with each face which implies the polyhedron is the actual shape and not an approximation.

For the sake of this exercise here are the normals where I take the cube to be an approximation of a sphere.